

En muchos casos, los programas requieren almacenar datos de forma persistente, de manera que estos datos se mantengan entre distintas ejecuciones del programa.

Los archivos son el mecanismo más básico y ampliamente utilizado en cualquier lenguaje de programación para lograr esta persistencia de datos.

Los archivos difieren significativamente de las bases de datos en el sentido de que son universales, lo que significa que cualquier programa que comprenda el formato de los datos almacenados en un archivo puede leerlos e incluso escribir sobre ellos, sin importar qué programa los haya generado previamente.

Existen notables diferencias entre los archivos y las bases de datos. Las bases de datos ofrecen ventajas significativas en términos de velocidad y capacidad de manipulación de grandes cantidades de datos. Además, proporcionan características de seguridad, como mecanismos de autenticación y autorización, para gestionar datos sensibles. Otro aspecto destacado de las bases de datos es su capacidad para mantener la consistencia de los datos, evitando la presencia de información incoherente o inconsistente.

A pesar de estas ventajas, la manipulación de datos a través de archivos ha sido ampliamente utilizada históricamente para transmitir información entre sistemas heterogéneos. Esta práctica es común cuando se requiere transferir datos de un programa a otro que no puede comunicarse directamente entre sí. Los archivos permiten que un programa almacene datos en un archivo y que otro programa, incluso escrito en un lenguaje o sistema diferente, pueda

manipular esos datos. Este mecanismo ha sido tradicionalmente utilizado y sigue siendo utilizado en el campo de la ciencia de datos, donde se requiere la transferencia y manipulación conjunta de datos provenientes de diferentes fuentes.

3.1 Tipos de archivos

Cuando se requieren manipular datos almacenados en archivos, es fundamental considerar el formato o tipo de archivo. Existen dos tipos principales de archivos: archivos de texto y archivos binarios.

Los archivos de texto contienen información representada como una secuencia de caracteres legibles, escritos en una determinada codificación (UTF-8, ASCII, etc.). Estos archivos utilizan caracteres especiales, como el retorno de carro o la nueva línea, para delimitar cada línea y permitir la manipulación individual de cada dato. El formato de archivo de texto es adecuado para almacenar datos legibles por humanos, como texto plano, configuraciones o registros de datos en formato tabular.

Por otro lado, los archivos binarios no dependen de los caracteres de retorno de carro para delimitar los datos. En cambio, requieren de algún mecanismo específico para indicar la forma en que los datos están organizados dentro del archivo.Los archivos binarios son archivos en los que la información se almacena en un formato no legible directamente por los humanos y pueden contener datos de diversos tipos, como números, imágenes, audio, video, entre otros, donde la representación en formato de texto no sería práctica o eficiente. A diferencia de los archivos de texto, los archivos binarios no están codificados como caracteres legibles, sino que utilizan una representación binaria de los datos.

En resumen, los archivos de texto son adecuados para almacenar datos en formato de texto legible, mientras que los archivos binarios son utilizados para guardar datos más complejos y diversos, sin requerir la conversión a cadenas de texto.

3.2 Apertura y cierre

Para interactuar con un archivo en Python, es necesario informar al sistema operativo qué archivo se pretende utilizar, dónde está ubicado y qué operaciones se van realizar en él. Para lograr esto, se debe realizar la operación

de apertura del archivo, indicando al sistema operativo que el programa está utilizando ese archivo y, potencialmente, evitando que otros programas accedan a él simultáneamente. Al finalizar el uso del archivo, es importante cerrarlo.

El cierre de un archivo cumple dos propósitos básicos. En primer lugar, notifica al sistema operativo que ya no se está utilizando y, por lo tanto, otros programas pueden acceder a él. En segundo lugar, en el caso de operaciones de escritura de datos en el archivo, el cierre garantiza que las últimas escrituras se almacenen correctamente en el disco o en el medio de almacenamiento utilizado. Al realizar escrituras, es probable que la biblioteca de manipulación de archivos o el propio sistema operativo espere a acumular varias operaciones de escritura antes de almacenarlas en el dispositivo de almacenamiento. Si un archivo no se cierra correctamente, es posible que las últimas escrituras no se envíen al destino, por lo que es importante cerrar siempre los archivos.

Python nos proporciona dos funciones para estas operaciones, llamadas precisamente `open` y `close`. La función open recibe dos parámetros. El primero es el nombre del archivo, una cadena que contiene el nombre y la extensión del archivo, y opcionalmente la ubicación utilizando rutas relativas o absolutas para especificar el directorio donde se encuentra el archivo. El segundo parámetro es el modo de apertura, donde se indica el tipo de archivo (texto o binario) y la operación que deseamos realizar en dicho archivo.

La función `open()` devuelve un objeto archivo que nos permite operar con el archivo. Este objeto cuenta con un método llamado `close()`, que se utiliza para indicar que el archivo ha sido cerrado correctamente.

En general, es recomendable mantener los archivos abiertos el menor tiempo posible. Por lo tanto, se debe abrir el archivo, realizar las operaciones necesarias y cerrarlo inmediatamente. No debe dejarse un archivo abierto más tiempo del necesario, especialmente durante el ingreso de datos por parte del usuario.

3.3 Modos de apertura

El modo de apertura que se especifica como parámetro en la función open es una cadena de texto que puede contener una o dos letras. Estas letras indican la operación que se desea realizar en el archivo. Para la operación de lectura, se utiliza la letra “R” (read), mientras que para la operación de escritura se utiliza la letra “W” (write).

El modo de apertura “W” tiene la particularidad de que, si el archivo en el que se desea escribir no existe, se crea automáticamente. Sin embargo, si el archivo ya existe, su contenido se borra por completo y se empieza a escribir desde cero, es decir, como un archivo vacío.

Existe otro modo de apertura para escritura denominado “X”. Este modo permite escribir datos en un archivo, pero si el archivo ya existe, impide borrar su contenido original y arroja un error. En resumen, el modo “X” es utilizado para escribir en un archivo completamente nuevo sin sobrescribir uno existente.

Otro modo de escritura es el modo “A” (append), que significa agregar. Cuando se abre un archivo en modo “A”, si el archivo no existe, se crea. Sin embargo, si el archivo ya existe, el contenido previo no se borra, sino que se comienza a escribir a continuación del último dato almacenado en el archivo.

Por último, existe el modo “R+” que permite tanto la lectura como la escritura en un archivo. Al abrir un archivo en modo “R+”, todas las operaciones de lectura y escritura se realizan desde el principio del archivo, aunque con operaciones de desplazamiento se puede indicar una posición específica para continuar a partir de ella.

Es importante mencionar que también se debe indicar el tipo de archivo con una letra “T” para archivos de texto y una letra “B” para archivos binarios. Si no se especifican estas letras, el modo de apertura predeterminado es de lectura de archivos de texto (“RT”).

Modo	Descripción	Acción sobre el archivo
`r`	Lectura	Abre el archivo para lectura (modo texto)
`w`	Escritura	Abre el archivo para escritura (modo texto)
`x`	Escritura exclusiva	Abre el archivo para escritura exclusiva (modo texto)
`a`	Adjuntar	Abre el archivo para agregar contenido al final (modo texto)
`r+`	Lectura y escritura	Abre el archivo para lectura y escritura (modo texto)
`rb`	Lectura binaria	Abre el archivo para lectura (modo binario)
`wb`	Escritura binaria	Abre el archivo para escritura (modo binario)
`xb`	Escritura exclusiva binaria	Abre el archivo para escritura exclusiva (modo binario)
`ab`	Adjuntar binario	Abre el archivo para agregar contenido al final (modo binario)
`r+b`	Lectura y escritura binaria	Abre el archivo para lectura y escritura (modo binario)

3.4 Lectura de archivos de texto

Para leer archivos de texto, hay tres métodos disponibles en la estructura devuelta por la función open. El primer método es `read`, que lee todo el contenido del archivo y devuelve una variable de tipo string. Este método es adecuado para archivos de texto relativamente pequeños, ya que carga todo el contenido en una sola variable. La principal desventaja es que, al leer todo el contenido, es responsabilidad del programador manipular el formato si el archivo tiene una estructura interna. Por ejemplo, si el archivo está separado por comas, será responsabilidad del programa dividir los datos según los delimitadores. Si el archivo contiene varias líneas de texto, read las lee todas y devuelve una cadena que incluye los caracteres de nueva línea o retorno de carro que indican el final de cada línea.

El segundo método es `readline`, que lee una sola línea y deja el archivo en

un estado en el que la siguiente llamada leerá la línea que se encuentre a continación. De esta manera, readline devuelve una cadena con el contenido de una línea y permite la lectura en un ciclo para procesar líneas sucesivas. Este es el método más adecuado cuando se trata de archivos relativamente grandes, ya que al leer las líneas una a una, no se carga todo el contenido del archivo en la memoria, sino que se mantiene una única línea en memoria a la vez.

Por último, el método `readlines` (en plural) recorre todo el archivo y lee cada una de las líneas, devolviendo una lista donde cada posición contiene una línea de texto. Si se lo utiliza en un archivo con, por ejemplo, 20 líneas, el mismo devolverá una lista con 20 elementos, cada uno representando una línea del archivo en el orden en que fueron leídas. Readlines se encarga de dividir el archivo según los caracteres de nueva línea, pero ocupa más memoria, ya que todo el contenido del archivo se almacena en la lista. Por lo tanto, no sería apropiado para leer archivos de texto demasiado grandes, en el orden de varios megabytes, ya que consumiría una cantidad considerable de memoria.

Por lo tanto, para leer todo el contenido de un archivo de texto en una variable e imprimirla, se puede programar el siguiente bloque:

```
archivo = open("datos.txt")
contenido = archivo.read()
print(contenido)
archivo.close()
```

Y para leer un archivo más largo linea a linea:

```
archivo = open("datos.txt")

linea = archivo.readline()
while linea:
    print(linea)
    linea = archivo.readline()

archivo.close()
```

En este ejemplo, se abre el archivo “datos.txt” en modo lectura. Luego se utiliza un ciclo while para leer cada línea del archivo utilizando el método readline(). Dentro del ciclo, se imprime la línea y se lee la siguiente línea. El ciclo continúa hasta que se alcanza el final del archivo, es decir, cuando el método readline() devuelve una cadena vacía. Por último, se cierra el archivo

utilizando el método close().

3.5 Escritura de archivos de texto

Para la escritura de archivos de texto existen dos métodos principales. El primer método es `write`, que permite escribir una cadena en el archivo. Al utilizar este método, la cadena se escribe en el archivo sin incluir automáticamente caracteres de nueva línea. Si se necesita escribir varias líneas de texto, es responsabilidad del programador agregar los caracteres de nueva línea en los lugares adecuados dentro de la cadena.

El segundo método es `writelines`, que recibe una lista de cadenas y escribe cada una de ellas en el archivo de texto en el orden en que aparecen en la lista. Al igual que el método write,writelines no agrega automáticamente caracteres de nueva línea, por lo que es responsabilidad del programador asegurarse de que las cadenas de la lista incluyan los caracteres de nueva línea donde sea necesario.

Por ejemplo:

```
nombres = ['Juan', 'María', 'Carlos', 'Laura']

archivo = open("nombres.txt", "w")

for nombre in nombres:
    archivo.write(nombre + '\n')

archivo.close()
```

En este ejemplo, se crea una lista llamada `nombres` con algunos nombres. Luego se abre el archivo “nombres.txt” en modo escritura. A continuación, se utiliza un bucle for para recorrer cada elemento de la lista nombres. En cada iteración, se utiliza el método write() para escribir el nombre en el archivo, seguido de un carácter de nueva línea `\\n`. Esto garantiza que cada nombre se escriba en una línea nueva del archivo.

3.6 Ejercicios

3.6.1 Lectura de un archivo de números

Desarrollar un programa que lea todo el contenido del archivo numeros.txt que contiene múltiples líneas de texto, cada una de ellas con un número

entero. Al leer el archivo se debe almacenar todos los números en una lista. A continuación el programa debe manipular los números cargados en la lista para:

- Calcular e imprimir el promedio de todos los números
- Calcular e imprimir la cantidad de números mayores al promedio
- Generar y mostrar una nueva lista que contenga todos los números pares

proceso_listas.py

```
def cantidad_menor(lista, techo):
    """
    Cuenta los elementos de una lista que sean menores a un tope

    :param lista: lista de valores
    :type lista: lista de datos que soporte comparación por menor
    :param techo: valor máximo para filtrar
    :type techo: el mismo de los elementos de la lista
    :returns: la cantidad de elementos cuyo valor sea menor al techo
    :rtype: entero
    """
    c = 0
    for x in lista:
        if x < techo:
            c += 1

    return c

def cantidad_mayor(lista, piso):
    """
    Cuenta los elementos de una lista que sean menores a un tope

    :param lista: lista de valores
    :type lista: lista de datos que soporte comparación por menor
    :param techo: valor máximo para filtrar
    :type techo: el mismo de los elementos de la lista
    :returns: la cantidad de elementos cuyo valor sea menor al techo
    :rtype: entero
    """
    c = 0
    for x in lista:
        if x > piso:
            c += 1

    return c


def promedio(lista):
    """
    Calcula el promedio simple de todos los elemento de una lista

    :param lista: lista
    :type lista: lista de números
    :returns: el promedio de todos los valores de la lista
              o 0 si la lista está vacía
    :rtype: flotante
```

```
    """
    cantidad = len(lista)
    if cantidad == 0:
        return 0

    suma = 0
    for x in lista:
        suma += x

    return suma / cantidad


def existe(lista, buscado):
    """
    Busca un valor en una lista e informa si lo pudo encontrar

    :param lista: una lista de cualquier tipo
    :param buscado: el valor que se busca
    :tipo buscado: el mismo tipo que el de los datos de la lista
    :returns: verdadero si el elemento buscado existe en la lista
              y falso en caso contrario
    :rtype: boolean
    """
    for x in lista:
        if x == buscado:
            return True

    return False
```

lectura_numeros.py

```
from proceso_listas import *

def leer_archivo(nombre_archivo):

    numeros = []

    archivo = open(nombre_archivo)

    while True:
        linea = archivo.readline()
        if not linea:
            break
        n = int(linea[:-1])
        numeros.append(n)

    archivo.close()

    return numeros


if __name__ == "__main__":
    lista = leer_archivo("numeros.txt")
    promedio_todos = promedio(lista)
    mayores = cantidad_mayor(lista, promedio_todos)
    pares = [x for x in lista if x % 2 == 0]

    print("Del archivo se leyeron los números")
```

```
    print(f"El promedio de esos números es de {promedio_todos:5.2f}")
    print(f"Y {mayores} son mayores que el promedio")
    print("Listado de los pares:")
    for x in pares:
        print(x)
```

3.6.2 Lectura de un csv con códigos postales

El archivo cp.csv contiene el listado de los códigos postales de tres provincias argentinas. En cada línea se encuentran, separados por un símbolo de punto y coma (;) los siguientes datos:

- Provincia: representada por una letra mayúscula según el estándar ISO correspondiente.
- Código: es un número entero de cuatro dígitos que identifica una localidad o varias localidades vecinas. Puede estar repetido.
- Nombre: es el nombre de la localidad correspondiente a ese código.

Se requiere leer todo el contenido del archivo y guardarlo en una lista que contenga un elemento por cada línea. En cada elemento debe almacenarse alguna estructura de datos que permita acceder individualmente a cada dato que conforma un codigo postal.

Luego de la carga el programa debe permitir el ingreso de uno o más códigos numéricos y listar la provincia y nombre de todas las localidades asignadas a dichos códigos

cp.csv (primeras líneas)

```
K;4743;ACONQUIJA
K;4701;ACOSTILLA
K;4234;ACHALCO
K;5263;ADOLFO E. CARRANZA
K;4139;AGUA AMARILLA (LA HOYADA, DPTO. SANTA MARIA)
```

lectura_cp.py

```python
def leer_codigos(nombre_archivo):

    codigos = []
    archivo = open(nombre_archivo)

    for linea in archivo.readlines():
        codigo = tuple(linea[:-1].split(";"))
        codigos.append(codigo)

    archivo.close()

    return codigos


def buscar_codigo(lista, buscado):
    encontrados = []

    for c in lista:
        if c[1] == buscado:
            encontrados.append(c)

    return encontrados
    # Con comprensión de listas
    # return [c for c in lista if c[1] == buscado]

if __name__ == "__main__":
    codigos = leer_codigos("cp.csv")

    buscado = input("Ingrese un código a buscar (fin con cadena vacía):")
    while buscado:
        encontrados = buscar_codigo(codigos, buscado)
        if encontrados:
            for c in encontrados:
                print(f"{c[0]}: {c[2]}")
        else:
            print("No se encontró ninguna localidad")
        buscado = input("Ingrese un código a buscar (fin con cadena vacía):")
```

3.6.3 Análisis del archivo de propinas

El archivo tips.csv contiene los datos de las propinas recibidas por los empleados de una pizzería en New York.

El archivo contiene los siguientes campos separados por comas

- total_bill: Importe total del servicio
- tip: Importe de la propina
- sex: Sexo del cliente
- smoker: El cliente es fumador (Yes, No)
- day: Día (Juev, Vie, Sab, Dom)
- time: Horario (Almuerzo, Cena)
- size: Tamaño de la mesa (comensales)

Se requiere desarrollar un programa que desde los datos del archivo informe:

- ¿Quién paga más propinas?¿Hombres o mujeres?
- ¿Cuáles son los días más 'lentos', con menos propinas?
- ¿Cuál es el promedio de las propinas?
- ¿Cuándo se vendió la orden más grande (qué día y en qué turno)?

tips.csv (primeras líneas)

```
total_bill,tip,sex,smoker,day,time,size
16.99,1.01,Mujer,No,Dom,Cena,2
10.34,1.66,Hombre,No,Dom,Cena,3
21.01,3.5,Hombre,No,Dom,Cena,3
23.68,3.31,Hombre,No,Dom,Cena,2
```

tips.py

```
propinas_hombres = propinas_mujeres = 0
propina_fumador = propina_no_fumador = 0
propinas_acum = [0]*4
promedio = 0
cant_operaciones = 0
dia_orden_mayor = None
turno_orden_mayor = None
monto_orden_mayor = None

#######################################################################
# Para utilizar la librería mejorada de consola (rich) primero hay que
# instalarla utilizando el comando pip install rich
#######################################################################
from rich import print
from rich.console import Console
from rich.panel import Panel

c = Console()
c.clear()

first_row = True
with open("tips.csv","r") as file:
    for fila in file:
        # Debo saltear el primer registro ya que contiene los titulos
        if first_row:
            # Indicar que ya no estoy en la primera fila
            first_row = False
        else:
            cant_operaciones += 1
            # Hacer todo el procesamiento
            # Ahora debo separar la fila leída en sus componentes
            campos = fila[:-1].split(",")
            # print(campos)
            # Saco los campos que necesito
            total = float(campos[0])
            propina = float(campos[1])
            sexo = campos[2]
            dia = campos[4]
            horario = campos[5]

            # Acumular por separado las propinas de hombres y mujeres
            if sexo == 'Hombre':
                propinas_hombres += propina
            else:
                propinas_mujeres += propina

            # Obtener los datos de la mayor orden
            # Cuándo? Cuando sea la primera o sea mayor a la anterior
            if monto_orden_mayor == None or total > monto_orden_mayor:
                monto_orden_mayor = total
                dia_orden_mayor = dia
                turno_orden_mayor = horario

            # Solo trabajo 4 días así que puedo acumular las propinas
            # en un vector de 4 elementos, relacionados
            # a Jue, Vie, Sab, Dom
            if dia == 'Juev':
                propinas_acum[0] += propina
            elif dia == 'Vie':
                propinas_acum[1] += propina
```

```
            elif dia == 'Sab':
                propinas_acum[2] += propina
            else:
                propinas_acum[3] += propina

# Calcular la propina promedio
promedio = (propinas_hombres+propinas_mujeres)/cant_operaciones

# Recorrer el vector de propinas y ver cual es el menor
menor_indice = 0  # Voy a suponer que es el primero
menor_propina = propinas_acum[0]
menor_dia = ""

for i in range(len(propinas_acum)):
    if propinas_acum[i] < menor_propina:
        menor_propina = propinas_acum[i]
        menor_indice = i
# Cuando termine tengo que ver a qué dia corresponde ese índice
if menor_indice == 0:
    menor_dia = "Jueves"
elif menor_indice == 1:
    menor_dia = "Viernes"
elif menor_indice == 2:
    menor_dia = "Sábado"
else:
    menor_dia = "Domingo"

c.print(Panel("Análisis de propinas en New York"))

print (f"[white bold on green]Propinas de los hombres:[/] {propinas_hombres:.2f}")
print (f"[white bold on green]Propinas de las mujeres:[/] {propinas_mujeres:.2f}")
print (f"[white bold on green]Propina promedio:[/] {promedio:.2f}")
print (f"La mayor venta fué de [on green]{monto_orden_mayor}[/] y se realizó un [on
↪  green]{dia_orden_mayor}[/] en el turno [on green]{turno_orden_mayor}[/]")
print (f"Los días más lentos son los [on green]{menor_dia}[/] con solamente [on
↪  green]{menor_propina}[/] de propina")
c.print(Panel("Fin de la ejecución"))
```



4.1 Conjuntos

4.1.1 Introducción

Los conjuntos son una estructura de datos proporcionada por el lenguaje que permite almacenar y manipular colecciones de elementos. A diferencia de las listas o las tuplas, los conjuntos tienen características distintivas que los hacen únicos.

Una de las principales características de los conjuntos es su capacidad para eliminar automáticamente elementos duplicados. Esto significa que, al insertar un elemento repetido en un conjunto, este será ignorado y no se agregará nuevamente.

Otra diferencia importante es que los conjuntos no admiten acceso indexado. No es posible acceder a elementos individuales utilizando el operador de corchetes o rebanadas, como se puede hacer en las listas o tuplas.

Sin embargo, la característica principal de los conjuntos es su eficiencia en las operaciones de búsqueda. Mientras que en las listas o tuplas una búsqueda puede requerir recorrer todos los elementos, en los conjuntos la verificación de pertenencia tiene complejidad $O(1)$ en promedio. Esto permite localizar rápidamente un elemento sin recorrer toda la colección.

Esta propiedad hace que los conjuntos sean especialmente útiles para realizar búsquedas de elementos sin duplicados. Además, los conjuntos son ideales para realizar operaciones de conjuntos matemáticos, como uniones, intersec-

ciones y diferencias. Estas operaciones, que requerirían una programación relativamente compleja y lenta con listas o tuplas, pueden ser realizadas de manera eficiente y sencilla con conjuntos.

En resumen, los conjuntos son una valiosa estructura de datos que permiten almacenar y manipular colecciones de elementos sin duplicados. Su eficiencia en las operaciones de búsqueda y su capacidad para realizar operaciones de conjuntos de manera rápida los convierten en una herramienta poderosa para resolver diversos problemas.

4.1.2 Operaciones de datos

Creación

Existen tres formas de crear conjuntos en Python. La primera opción es utilizar la función `set()`, que crea un conjunto vacío. Este conjunto vacío se puede asignar a una variable para su posterior manipulación. Una vez creado el conjunto vacío, se pueden utilizar métodos para insertar elementos en él.

La segunda opción es crear un conjunto mediante la enumeración de elementos conocidos. Esto se asemeja a la notación matemática, donde se definen los elementos del conjunto entre llaves y separados por comas. Por ejemplo: `{1, 2, 3}`.

La tercera opción es utilizar comprensiones de conjuntos, que son similares a las comprensiones de listas. Se utiliza la misma sintaxis de comprensión, pero se delimita con un par de llaves en lugar de un par de corchetes. En ambos casos, es importante tener en cuenta que los conjuntos no permiten elementos duplicados, por lo que la cantidad de elementos en el conjunto resultante puede ser menor a la cantidad de elementos especificados en la creación. Los elementos duplicados serán ignorados. Por ejemplo: `{random.randint(1,10000) for x in range(100)}` crea un conjunto con hasta 100 números al azar sin repeticiones.

Inserción

Una vez que se ha creado un conjunto utilizando una de las tres alternativas mencionadas anteriormente, es posible agregar nuevos valores al conjunto utilizando dos métodos específicos.

El primer método es `add()`, que permite agregar un nuevo elemento al conjunto verificando que no exista previamente. En caso de que el elemento ya esté presente en el conjunto, el método add() no generará ninguna acción

ni devolverá ningún error. El conjunto permanecerá sin modificaciones. Sin embargo, si el elemento que se intenta agregar no se encuentra en el conjunto, el método add() lo añadirá correctamente, expandiendo el conjunto con un elemento adicional.

El segundo método es `update()`, el cual recibe como parámetro una secuencia iterable, como un rango, una cadena, una tupla, una lista o incluso otro conjunto. Este método se encarga de recorrer todos los elementos de la secuencia iterable y los inserta en el conjunto original. Sin embargo, al igual que el método add(), el método update() garantiza que solo se insertarán elementos que no estén duplicados y que no existan previamente en el conjunto. Esto significa que no se agregarán elementos repetidos al conjunto durante el proceso de actualización.

Eliminación

Además de agregar elementos a un conjunto, también es posible eliminar elementos utilizando varios métodos disponibles.

El primer método es `remove()`, el cual recibe como parámetro el valor que se desea eliminar del conjunto. Este método elimina el elemento especificado y, en caso de que el elemento no exista en el conjunto, generará un error. Es importante tener cuidado al utilizar remove() para asegurarse de que el elemento a eliminar realmente esté presente en el conjunto.

El segundo método es `discard()`, que también recibe como parámetro el valor que se desea eliminar. Si el elemento existe en el conjunto, discard() lo eliminará sin generar errores. Sin embargo, si el elemento no está presente, discard() simplemente ignorará la solicitud y permitirá que el programa continúe sin interrupciones.

Otra opción para eliminar elementos es el método `pop()`, el cual selecciona y elimina arbitrariamente un elemento del conjunto. El elemento eliminado se devuelve como resultado de la llamada a pop(), lo que permite al usuario conocer qué elemento fue eliminado en caso de necesitarlo.

Por último, el método `clear()` borra todos los elementos del conjunto, dejándolo vacío. Este método es útil cuando se desea reiniciar el conjunto y eliminar todos sus elementos sin eliminar la variable en sí.

Recorridos

Cuando se trabaja con conjuntos y se desea recorrer sus elementos, se puede utilizar la instrucción for de manera similar a como se recorren las listas. Con la sintaxis for variable in conjunto, se realizará una iteración por cada elemento almacenado en el conjunto, asignando cada valor a la variable del ciclo for.

Es importante tener en cuenta que al recorrer un conjunto, los elementos no se encontrarán en un orden específico. A diferencia de las listas, donde los elementos se mantienen en el orden en que se insertaron, un conjunto no preserva el orden de inserción.

Si se requiere recorrer un conjunto en un orden específico, como un orden numérico o alfabético, no es posible lograrlo directamente con un conjunto. Sin embargo, la función `sorted()` devuelve una nueva lista con los mismos elementos del conjunto, pero ordenados según sus valores. Es importante destacar que esto implica la creación de una nueva estructura de datos (la lista ordenada) y no afecta el orden interno del conjunto.

A veces, al recorrer un conjunto, puede parecer que los elementos están ordenados, especialmente si se insertan solo números enteros. Sin embargo, esto no se puede garantizar para otros tipos de datos o para conjuntos más grandes. Por lo tanto, nunca se debe confiar en que un conjunto tendrá los datos ordenados, a menos que se utilice el método sorted() para obtener una lista ordenada explícitamente.

4.1.3 Operaciones de conjuntos

Los conjuntos son especialmente útiles para eliminar elementos duplicados, ya que garantizan la unicidad de los datos almacenados. Además, permiten realizar operaciones de conjuntos similares a las que se utilizan en matemáticas.

Pertenencia

La operación principal en un conjunto es verificar la pertenencia de un elemento. Para esto, se utiliza el operador `in`, que verifica si un valor dado pertenece al conjunto. El operador in devuelve True si el elemento está presente en el conjunto y False en caso contrario. La diferencia principal con respecto a las listas es que la búsqueda en un conjunto tiene complejidad $O(1)$ en promedio,

por lo que no es proporcional a la cantidad de elementos almacenados.

Cuando se trabaja con más de un conjunto, se pueden realizar operaciones de conjuntos, como la unión, la intersección y la diferencia. Estas operaciones se aplican utilizando diferentes métodos.

Unión

La unión de conjuntos se puede lograr mediante el método `union()`. Este método recibe como parámetro un segundo conjunto y devuelve un nuevo conjunto que contiene todos los elementos no repetidos de ambos conjuntos. También está disponible el método update(), que realiza la unión modificando el conjunto original.

Intersección

La intersección de conjuntos se puede lograr utilizando el método `intersection()`. Este método recibe como parámetro un segundo conjunto y devuelve un nuevo conjunto con los valores que existen en ambos conjuntos. Además, el método `intersection_update()` realiza la intersección modificando el conjunto original.

Diferencias

La diferencia de conjuntos se puede obtener mediante el método `difference()`. Este método se aplica a un conjunto y recibe como parámetro otro conjunto. Retorna un nuevo conjunto con los elementos presentes en el primer conjunto pero no en el segundo. También está disponible el método `difference_update()`, que realiza la diferencia modificando el conjunto original.

La diferencia simétrica, también conocida como diferencia exclusiva, se obtiene utilizando el método `symmetric_difference()`. Este método recibe como parámetro un segundo conjunto y devuelve un nuevo conjunto que contiene todos los elementos presentes en ambos conjuntos, excepto aquellos que existen en ambos conjuntos al mismo tiempo. El método `symmetric_difference_update()` realiza la diferencia simétrica modificando el conjunto original.

4.1.4 Casos de aplicación

4.1.5 Eliminación de repetidos de una lista

En este ejemplo se dispone de una lista llamada `lista` que contiene elementos duplicados. Para eliminar los duplicados, se convierte la lista en un conjunto

utilizando la función `set()`. Los conjuntos en Python sólo pueden contener elementos únicos, por lo que al convertir la lista en un conjunto, automáticamente se eliminan los elementos duplicados.

Luego se convierte nuevamente el conjunto a una lista utilizando la función `list()`y se asigna en una nueva lista llamada `lista_sin_duplicados` que contiene los elementos de la lista original sin duplicados.

Finalmente, se imprime la lista sin duplicados para verificar el resultado.

```
# Lista con elementos duplicados
lista = [1, 2, 3, 4, 2, 3, 5, 6, 1, 7, 8, 5, 9]

# Convertir la lista en un conjunto
# para eliminar duplicados
conjunto = set(lista)

# Convertir el conjunto nuevamente a una lista
lista_sin_duplicados = list(conjunto)

# Imprimir la lista sin duplicados
print(lista_sin_duplicados)

#[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

4.1.6 Operaciones entre múltiples conjuntos

En este ejemplo se dispone de dos conjuntos `curso_a` y `curso_b` que representan los nombres de personas inscriptas en dos cursos diferentes y sobre ellos se utilizan las siguientes operaciones de conjuntos:

La operación de unión (`union()`) combina ambos conjuntos y devuelve un nuevo conjunto con todos los elementos únicos de ambos conjuntos.

La operación de intersección (`intersection()`) encuentra los elementos comunes entre ambos conjuntos y devuelve un nuevo conjunto con esos elementos.

La operación de diferencia (`difference()`) encuentra los elementos que están en el primer conjunto pero no en el segundo conjunto y devuelve un nuevo conjunto con esos elementos.

La operación de diferencia simétrica (`symmetric_difference()`) encuentra los elementos que están en uno de los conjuntos pero no en ambos conjuntos y devuelve un nuevo conjunto con esos elementos.

Cada operación se aplica a los conjuntos y el resultado se almacena en una variable correspondiente. A continuación se imprimen los conjuntos resultantes para ver los nombres de las personas según cada operación.

```python
# Conjunto de nombres de personas del curso A
curso_a = {"Juan", "María", "Pedro", "Luisa", "Ana"}

# Conjunto de nombres de personas del curso B
curso_b = {"Pedro", "Ana", "Sofía", "Carlos"}

# Unión: personas inscriptas en al menos uno de los cursos
union = curso_a.union(curso_b)
print(union)
# {'Juan', 'Carlos', 'María', 'Pedro', 'Luisa', 'Ana', 'Sofía'}

# Intersección: personas inscriptas en ambos cursos
interseccion = curso_a.intersection(curso_b)
print(interseccion)
# {'Ana', 'Pedro'}

# Diferencia: inscriptos sólo en el curso A
diferencia = curso_a.difference(curso_b)
print(diferencia)
# {'María', 'Luisa', 'Juan'}

# Diferencia simétrica: inscriptos en
# uno de los cursos, pero no en ambos
diferencia_simetrica = curso_a.symmetric_difference(curso_b)
print(diferencia_simetrica)
# {'Carlos', 'Sofía', 'María', 'Juan', 'Luisa'}
```

4.2 Diccionarios

4.2.1 Introducción

Los diccionarios son una estructura de datos altamente versátil y poderosa que permite almacenar información de manera distinta a otras estructuras disponibles en el lenguaje. A diferencia de estas, los diccionarios no almacenan conjuntos o grandes volúmenes de datos, sino que se basan en pares de datos conocidos como “clave” y “valor”. También denominados “arreglos asociativos” en otros lenguajes, los diccionarios establecen una relación entre una clave única en todo el conjunto y un valor asociado a esa clave.

En esencia, los diccionarios almacenan pares de datos, donde la clave actúa como identificador único para un dato específico, y el valor corresponde al dato asociado a esa clave. Este concepto es similar al de las claves primarias utilizadas en estructuras de bases de datos relacionales. Una característica destacada de los diccionarios es la velocidad de las operaciones de búsqueda: en promedio tienen complejidad $O(1)$, incluso cuando el diccionario contiene una gran cantidad de datos.

Los diccionarios no tienen un índice numérico para acceder a sus elementos y no permiten claves duplicadas. Desde Python 3.7 preservan el orden

en que se insertan sus pares clave-valor, aunque su principal ventaja sigue siendo la rapidez para recuperar un valor a partir de su clave.

Los diccionarios son especialmente útiles en situaciones en las que es necesario asociar un dato con un valor identificador. Por ejemplo, al almacenar datos de nombres de personas, es posible utilizar un diccionario en el que los nombres estén asociados con los números de documento de cada individuo. De esta manera, al proporcionar el número de documento al diccionario, se obtendría el nombre correspondiente asociado a ese documento.

Si bien es posible lograr una funcionalidad similar utilizando arreglos y utilizando el valor clave como índice del arreglo, en muchos casos el valor clave no puede ser equiparado a un índice de arreglo. Por ejemplo, si se requiere almacenar datos de personas y utilizar el número de documento como índice de un arreglo, se enfrentaría a la impracticabilidad de crear un arreglo con 100 millones de elementos para cubrir todas las posibilidades de números de documento de 8 dígitos. Los diccionarios, en cambio, permiten utilizar cualquier tipo de dato disponible en Python como clave, ya sean valores numéricos, cadenas u otros tipos de datos.

La flexibilidad de los diccionarios radica en su capacidad para establecer relaciones directas entre datos y valores identificadores, independientemente de su tipo o dominio. Esto los convierte en una herramienta poderosa para abordar una amplia gama de situaciones donde es necesario asociar y recuperar datos de manera eficiente.

4.2.2 Operaciones

Creación

La sintaxis utilizada para crear diccionarios es bastante similar a la de los conjuntos. Existen tres mecanismos principales para crear un diccionario.

El primero de ellos es utilizando la función `dict()`, la cual inicializa un diccionario vacío al que se le pueden ir agregando pares clave-valor.

Otra forma de crear un diccionario es mediante la enumeración explícita de sus datos. Se utiliza un par de llaves (`{}`) y se enumeran los valores separados por comas. Sin embargo, a diferencia de los conjuntos, en los diccionarios se debe establecer un par clave-valor para cada valor enumerado. Esto se realiza utilizando dos puntos (`:`) para asociar la clave con su respectivo valor.

Por último, se puede utilizar la comprensión de diccionarios para crear diccionarios de manera concisa. La sintaxis es similar a la comprensión de

conjuntos, utilizando un par de llaves que encierra una expresión de comprensión. En este caso, se establece el par clave-valor, donde tanto la clave como los valores pueden depender de los valores iterados en un ciclo for, utilizando dos puntos (:) para asociarla con el valor correspondiente.

Acceso

Una vez que se ha creado un diccionario, es posible acceder a sus valores utilizando el operador de corchetes (`[]`). En este caso, se indica la clave correspondiente al valor que se desea obtener. A diferencia de las listas, en los diccionarios la clave puede ser de cualquier tipo válido en Python. Por lo tanto, se puede utilizar el nombre del diccionario seguido del operador de acceso y, dentro de este, se especifica la clave que se busca. El acceso devuelve en promedio en tiempo $O(1)$ el valor asociado con la clave. Si la clave no existe en el diccionario, se produce una excepción `KeyError`.

Otra forma de acceder a los valores almacenados en el diccionario es utilizando el método `get()`. Este método recibe como parámetro la clave que se busca y, en caso de que la clave no se encuentre en el diccionario, devuelve None por defecto. Sin embargo, get() acepta un segundo parámetro opcional que permite especificar un valor por defecto. De esta manera, se puede acceder al valor asociado a la clave sin necesidad de utilizar una instrucción condicional, y se obtendrá el valor por defecto si la clave no existe en el diccionario.

Por último, existe un método llamado `setdefault()`, que recibe una clave y un valor por defecto como parámetros. Si la clave existe en el diccionario, el método devuelve el valor asociado a esa clave. En caso contrario, crea una nueva entrada en el diccionario asociando la clave con el valor por defecto especificado en el segundo parámetro. Este método es útil para evitar la verificación de existencia de la clave y asignar un valor predeterminado si no se encuentra en el diccionario.

Para poder conocer la longitud de un diccionario se dispone de la instrucción `len` que funciona de la misma manera que con todas las colecciones e iterables.

Inserción

Para agregar nuevos elementos o modificar elementos existentes en un diccionario, se utiliza el operador de acceso indexado de manera similar a como se haría con un arreglo. Es decir, se utiliza el nombre del diccionario seguido

del símbolo de corchetes con la clave deseada en su interior, y luego se utiliza el operador de asignación (=) para asignar un nuevo valor a esa clave.

Si la clave ya existe en el diccionario, el valor asociado a esa clave se actualizará con el nuevo valor asignado. En caso de que la clave no exista previamente en el diccionario, se creará una nueva entrada con la clave y el valor especificados. De esta manera, se puede agregar o modificar elementos en el diccionario de forma dinámica y flexible.

Borrado

Para eliminar elementos de un diccionario, existen dos métodos principales. El primero es utilizar la instrucción 'del', seguida del nombre del diccionario y entre corchetes la clave que se desea borrar. Esta instrucción simplemente elimina la clave y el valor asociado a ella.

El segundo método es el uso del método 'pop()'. Este método recibe como argumento la clave que se desea eliminar y devuelve el valor asociado a esa clave. Además, elimina la clave y su valor del diccionario. Si se proporciona un segundo argumento opcional al método 'pop()', se especifica un valor por defecto que será retornado si la clave no existe en el diccionario. Esto evita que se produzca un error en caso de que la clave no exista.

Otra opción es utilizar el método 'clear()', el cual borra todos los pares clave-valor del diccionario, dejándolo vacío como si estuviera recién creado. Este método no requiere ningún argumento.

Finalmente, la instrucción 'del' también puede utilizarse para eliminar todo el diccionario por completo, incluyendo sus valores contenidos, lo cual implica eliminar también la variable del diccionario en sí.

Estas opciones permiten la manipulación y eliminación de elementos en un diccionario de acuerdo a las necesidades específicas del programa.

Recorrido

Los diccionarios ofrecen tres formas de ser recorridos. El método `keys()` devuelve una vista con todas las claves: no hay claves repetidas y puede recorrerse o utilizarse en operaciones de conjuntos. Por otro lado, `values()` devuelve una vista con todos los valores; estos pueden repetirse. Las vistas reflejan los cambios realizados posteriormente en el diccionario.

También pueden recorrerse los pares ordenados (clave, valor) con un ciclo for. Pero en este caso el ciclo posee dos variables que se asignan en cada vuelta.

Para ello se dispone del método `items()`, que devuelve una vista de pares. En cada vuelta del for puede asignarse el par o directamente desempaquetarlo en dos variables. En el ejemplo siguiente se las desempaqueta en las variables numero y nombre:

```
diccionario = {1: "Lunes", 2: "Martes", 3: "Miércoles"}

for numero, nombre in diccionario.items():
    print(f"El día {numero} se llama {nombre}")
```



Excepciones y gestión de errores

5.1 ¿Qué son las excepciones?

Una excepción es una condición de error no prevista en un programa.

Tradicionalmente, en la programación estructurada, el programador debía verificar cada resultado de ejecución y detectar los posibles problemas.

Con la técnica de excepciones el programador debe establecer un bloque de código a ejecutar cuando una excepción se produzca y es el lenguaje el responsable de detectar una condición de ejecución anómala.

Esto genera un código más claro y un manejo de errores más compacto y ordenado.

5.2 Captura y disparo de Excepciones

Existen dos conceptos fundamentales en el manejo de excepciones, la captura (`except`) y el disparo (`raise`) de excepciones.

Durante la captura, el programador escribe código para que, al producirse una excepción de un cierto tipo, se puedan tomar las medidas necesarias, ya sea para corregir la condición de error, reportarla o proveerle al usuario medios alternativos para realizar la operación solicitada.

```
b = 10
c = 0
try:
```

```
    # Dividir dos números
    a = b / c
    print (a)
except ZeroDivisionError:
    # Se produce una excepción de división por cero
    print ("El divisor de la expresión es 0 (cero)")
```

Durante el disparo de excepciones el rol del programador es el contrario. Al detectar una condición errónea el programador notifica, por medio de algún tipo de excepción, a otros procesos, que dicha condición se ha producido y provee los datos necesarios para que dichos procesos puedan tomar conocimiento del error, sus posibles causas y tomar medidas al respecto.

```
b = 10
c = 0
# Validar si el divisor es 0 (cero)
if (c == 0):
    # En caso afirmativo disparar una excepción acorde
    raise ZeroDivisionError
```

5.3 Tipos de excepciones

Python dispone de diversos tipos de excepciones que indican el tipo de error producido. Estos son los más comunes:

TypeError : Ocurre cuando se aplica una operación o función a un dato del tipo inapropiado.

ZeroDivisionError : Ocurre cuando se intenta dividir por cero.

OverflowError : Ocurre cuando un cálculo excede el límite para un tipo de dato numérico.

IndexError : Ocurre cuando se intenta acceder a una secuencia con un índice que no existe.

KeyError : Ocurre cuando se intenta acceder a un diccionario con una clave que no existe.

FileNotFoundError : Ocurre cuando se intenta acceder a un fichero que no existe en la ruta indicada.

ImportError : Ocurre cuando falla la importación de un módulo.

Los módulos de terceros pueden definir sus propios tipos de excepciones, tal como se explicará en más detalle en una sección posterior.

5.4 Estructura de control de excepciones

Python utiliza un bloque `try-except-else`

```
try:
   # Bloque de código que puede generar
   # algún tipo de error
except _tipo1_:
   # Bloque que se ejecuta si se produce una excepción
   # de tipo _tipo1_
except _tipo2_:
   # Bloque que se ejecuta si se produce una excepción
   # de tipo _tipo2_
else:
   # Bloque que se ejecuta si NO SE
   # PRODUCE NINGUN ERROR
   # El bloque else es opcional
```

5.4.1 Ejemplo de gestión de excepciones

Este ejemplo intenta realizar una división. En caso de que el denominador sea cero se va a disparar una excepción, la cual en lugar de aparecerle al usuario, es capturada por el programa que tiene así la oportunidad de presentar un mensaje claro con respecto al error.

```
def division(a, b):
try:
    result = a / b
except ZeroDivisionError:
    print('¡No se puede dividir por cero!')
else:
    print(result)
```

En este caso se intenta abrir un archivo. Si el archivo no existe se dispara una excepción que es capturada y se informa el error. Esto permitiría la usuario, eventualmente, seleccionar un archivo diferente.

```
try:
   f = open('fichero.txt')  # El fichero no existe
except FileNotFoundError:
   print('¡El fichero no existe!')
else:
   print(f.read())
```

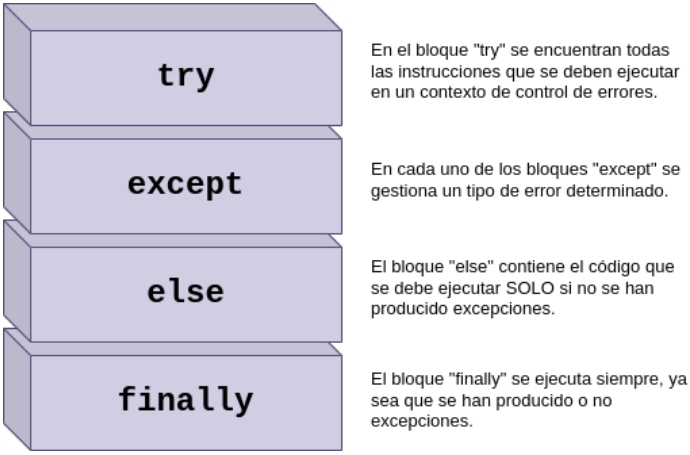


Figura 5.1: Estructura completa de un bloque de excepción

5.5 La estructura completa del bloque try-except

El bloque `try-catch` de Python tiene una estructura simple, conformada solamente por esas dos instrucciones, pero también tiene una estructura completa que nos permite realizar varias acciones más, tanto en caso de que se produzcan errores como en caso de que el programa funcione correctamente.

Las diferentes secciones del bloque tienen las siguientes funciones:

try Este bloque contiene todo el código a ejecutar en condiciones normales.

except Este bloque contiene el código que se debe ejecutar en respuesta a una excepción de un cierto tipo. Puede repetirse para manejar distintos tipos de excepciones.

else Este bloque contiene código que se ejecuta solamente cuando no se ha producido ninguna excepción.

finally Este bloque contiene código que se ejecuta, incondicionalmente, ya sea que se hayan producido o no excepciones como parte del flujo normal de ejecución.

5.6 Excepciones definidas por el programador

Es posible, en Python, que un programador defina y utilice sus propios tipos de excepciones, para mejorar la legibilidad del código, para expresar con mayor exactitud las condiciones de error de un programa o para considerar situaciones no cubiertas por los tipos de excepciones existentes en el lenguaje o en módulos de terceros.

Para ello debe crear una clase que herede de clase base `Exception`, la cual es la base de todas las excepciones gestionadas por Python.

```
# Definir una Excepción personalizada
class SalarioInsuficiente(Exception):
    """Se dispara cuando el salario del empleado
       es insuficiente.

    Atributos:
        salario -- salario que disparó el error
        message -- explicación del error
    """

    def __init__(self,
                 salario,
                 message="El salario no está entre (5000, 15000)."):
        self.salario = salario
        self.message = message
        super().__init__(self.message)
```

Una vez que esa excepción está definida, la misma se puede disparar como cualquier otra excepción estándar del lenguaje, mediante el comando `raise SalarioInsuficiente(1200)`.